

Project03 Wiki

Information

강의 명 : 운영체제 (12299)

과제 명 : ELE3021_Project #3

작성자 : 컴퓨터소프트웨어학부 2020003309 최성민

Design

Multi Indirect

- INDIRECT 관련 상수, FSSIZE 등을 조정해줍니다.
- inode 구조체와 dinode 구조체에 Double Indirect와 Triple Indirect 공간을 추가합니다. 이 공간은 각각 2 Level, 3 Level 트리구조로 형성됩니다.
- bmap 함수와 itrunc 함수에 Single Indirect 뿐만 아니라 Double Indirect와 Triple Indirect를 핸들링할 수 있도록 로직을 추가해줍니다.
- 기존의 Single Indirect 로직을 살려둡니다.

Symbolic Link

- 기존의 하드링크가 구현되어 있는 sys_link 시스템 콜 레포 함수를 참조하여 sys_symlink 함수를 구현합니다.
- ln 명령어가 -s, -h 옵션을 받을 수 있도록 ln 파일을 수정합니다.
- namex 함수에서 file path로 inode를 찾아줍니다. 해당 함수에서 symbolic link 파일을 만났을 경우를 핸들링해줍니다.

Sync

- 기존의 log.c와 bio.c 파일을 참조하여 sync 함수를 구현합니다.

- bflush 함수를 구현합니다. 이 함수는 buffer가 꽉 찼을 경우, buffer의 flush 가능한 Dirty buffer를 disk로 flush해주는 함수입니다.
- bfull 함수를 구현합니다. 이 함수는 unused buffer가 없는지 판별해주는 함수입니다.

Implement

Multi Indirect

- param.h

```
13 #define FSSIZE      2100000 // size of file system in blocks // 128*128*128 = 2097152
14 |
```

FSSIZE를 Triple Indirect까지 수용 가능하도록 크게 잡아줍니다.

- fs.h

```
23
24 #define NDIRECT 10
25 #define NINDIRECT (BSIZE / sizeof(uint)) // 128
26 #define DINDIRECT (NINDIRECT * NINDIRECT) // 128 * 128
27 #define TINDIRECT (DINDIRECT * NINDIRECT) // 128 * 128 * 128
28 #define MAXFILE (NDIRECT + NINDIRECT + DINDIRECT + TINDIRECT)
29
```

각 상수를 알맞게 바꿔줍니다.

이때, addrs에 Double Indirect와 Triple Indirect를 추가해주되 dinode 구조체의 크기를 보존해야 하므로, NDIRECT는 10으로 줄어듭니다.

```
29
30 // On-disk inode structure
31 struct dinode {
32     short type;           // File type
33     short major;         // Major device number (T_DEV only)
34     short minor;         // Minor device number (T_DEV only)
35     short nlink;         // Number of links to inode in file system
36     uint size;           // Size of file (bytes)
37     uint addrs[NDIRECT+3]; // Data block addresses
38 };
39
```

addrs[NDIRECT] : Single Indirect

addrs[NDIRECT + 1] : Double Indirect

addrs[NDIRECT + 2] : Triple Indirect

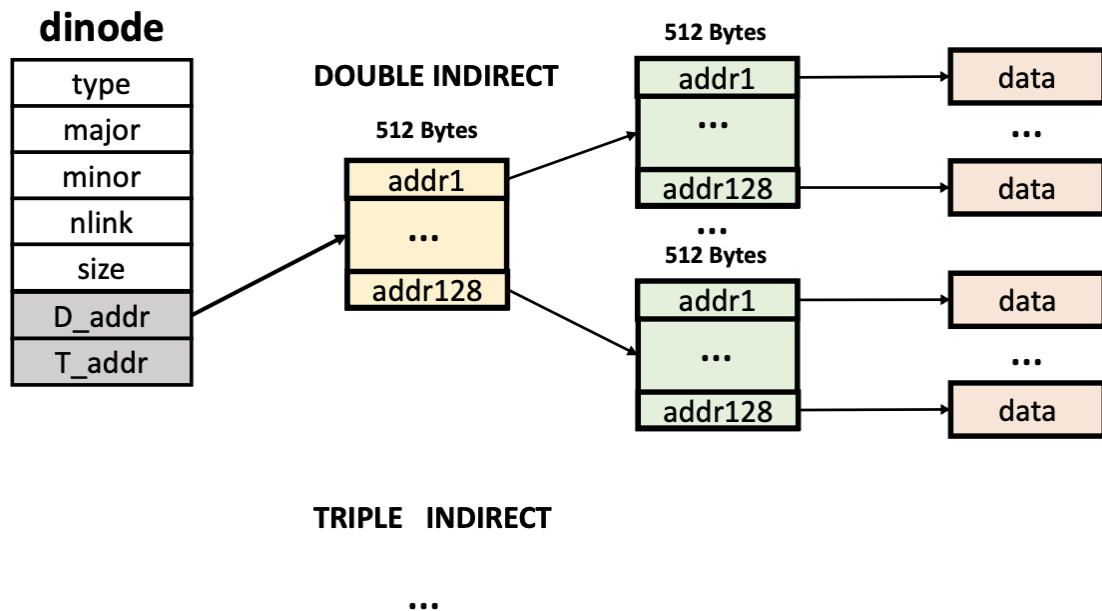
- file.h

```

12 // in-memory copy of an inode
13 struct inode {
14     uint dev;           // Device number
15     uint inum;          // Inode number
16     int ref;            // Reference count
17     struct sleeplock lock; // protects everything below here
18     int valid;          // inode has been read from disk?
19
20     short type;         // copy of disk inode
21     short major;
22     short minor;
23     short nlink;
24     uint size;
25     uint addrs[NDIRECT+3];
26 };
27

```

inode 또한 addrs의 구성이 dinode와 동일하다.



- fs.c

```

372
373 uint
374 get_block(struct inode *ip, uint addr, uint bn)
375 {
376     uint *a;
377     struct buf *bp;
378
379     bp = bread(ip->dev, addr);
380     a = (uint*)bp->data;
381     if((addr = a[bn]) == 0){
382         a[bn] = addr = balloc(ip->dev);
383         log_write(bp);
384     }
385     brelse(bp);
386
387     return addr;
388 }
389

```

bmap 함수의 길이를 단축하기 위한 get_block 함수입니다.

기존 bmap의 로직을 떼어서 만들었습니다.

```
390 static uint
391 bmap(struct inode *ip, uint bn)
392 {
393     uint addr;
394
395     if(bn < NDIRECT){
396         if((addr = ip->addrs[bn]) == 0)
397             ip->addrs[bn] = addr = balloc(ip->dev);
398         return addr;
399     }
400     bn -= NDIRECT;
401
402     // single
403     if(bn < NINDIRECT){
404         // Load indirect block, allocating if necessary.
405         if((addr = ip->addrs[NINDIRECT]) == 0)
406             ip->addrs[NINDIRECT] = addr = balloc(ip->dev);
407         addr = get_block(ip, addr, bn); // 1-level block found
408         return addr; // return found block
409     }
410     bn -= NINDIRECT;
```

bmap 함수입니다. single indirect 로직 안에 get_block 함수가 쓰였습니다.

```
411
412     // double
413     if (bn < DINDIRECT){
414         if((addr = ip->addrs[NINDIRECT + 1]) == 0)
415             ip->addrs[NINDIRECT + 1] = addr = balloc(ip->dev);
416
417         uint bn_2 = bn / NINDIRECT;
418         uint bn_1 = bn % NINDIRECT;
419
420         addr = get_block(ip, addr, bn_2); // 2-level
421         addr = get_block(ip, addr, bn_1); // 1-level
422         return addr;
423     }
424     bn -= DINDIRECT;
425
426     // triple
427     if (bn < TINDIRECT){
428         if((addr = ip->addrs[NINDIRECT + 2]) == 0)
429             ip->addrs[NINDIRECT + 2] = addr = balloc(ip->dev);
430
431         uint bn_3 = bn / DINDIRECT;
432         uint bn_2 = (bn - bn_3 * DINDIRECT) / NINDIRECT;
433         uint bn_1 = (bn - bn_3 * DINDIRECT) % NINDIRECT;
434
435         addr = get_block(ip, addr, bn_3); // 3-level
436         addr = get_block(ip, addr, bn_2); // 2-level
437         addr = get_block(ip, addr, bn_1); // 1-level
438         return addr;
439     }
440
441     panic("bmap: out of range");
442 }
443
```

이는 double과 triple indirect 로직에서도 비슷합니다.

해당 로직들은 트리 구조를 순서대로 따라가며 data block을 찾아 주소를 리턴합니다.

```
450 // N = Single Indirect
451 void
452 free_entry(struct inode *ip, uint addr, int n)
453 {
454     int i;
455     struct buf *bp;
456     uint *a;
457
458     bp = bread(ip->dev, addr);
459     a = (uint*)bp->data;
460
461     for(i = 0; i < NINDIRECT; i++){
462         if(a[i]){
463             if (n > 0) free_entry(ip, a[i], n-1); // recursive
464             else bfree(ip->dev, a[i]);
465         }
466     }
467     brelse(bp);
468     bfree(ip->dev, addr);
469 }
470
```

free_entry 함수입니다. 이 함수는 itrunc 함수의 길이를 줄이기 위해 만들어졌습니다.
재귀를 통해 제일 깊은 블록부터 차례대로 해제시켜나갑니다.

```
471 static void
472 itrunc(struct inode *ip)
473 {
474     int i;
475
476     for(i = 0; i < NDIRECT; i++){
477         if(ip->addrs[i]){
478             bfree(ip->dev, ip->addrs[i]);
479             ip->addrs[i] = 0;
480         }
481     }
482
483     for(i = 0; i < 3; i++){
484         if(ip->addrs[NDIRECT + i]){
485             free_entry(ip, ip->addrs[NDIRECT + i], i);
486             ip->addrs[NDIRECT + i] = 0;
487         }
488     }
489
490     ip->size = 0;
491     iupdate(ip);
492 }
493
```

itrunc 함수입니다.

이 함수는 N Direct부터 Triple Indirect 까지 차례대로 할당된 공간을 해제시킵니다.

Symbolic Link

- ln.c

```

5  int
6  main(int argc, char *argv[])
7  {
8      int flag = 0;
9
10     if(argc != 4){
11         printf(2, "Usage: ln [-s|-h] [old] [new]\n");
12         exit();
13     }
14
15     if (strcmp(argv[1], "-s") == 0) {
16         flag = symlink(argv[2], argv[3]);
17     } else if (strcmp(argv[1], "-h") == 0) {
18         flag = link(argv[2], argv[3]);
19     } else {
20         printf(2, "Usage: ln [-s|-h] [old] [new]\n");
21         exit();
22     }
23
24     if(flag < 0)
25         printf(2, "link %s %s: failed\n", argv[2], argv[3]);
26     exit();
27 }
28

```

ln 명령어를 위한 ln.c 파일입니다.

옵션에 따라 기호실행을 하기 위해 -s 와 -h 옵션을 식별하는 로직이 추가되었습니다.

하드링크의 경우 link 시스템콜을, 심볼릭링크의 경우 symlink 시스템콜을 호출합니다.

- sysfile.c

```

287  int
288  sys_symlink(void)
289  {
290      char *new, *old;
291      struct inode *ip;
292
293      if(argstr(0, &old) < 0 || argstr(1, &new) < 0)
294          return -1;
295
296      begin_op();
297
298      if((ip = create(new, T_SYM, 0, 0)) == 0){
299          end_op();
300          return -1;
301      }
302
303      if(writei(ip, old, 0, strlen(old) + 1) != strlen(old) + 1){
304          iunlockput(ip);
305          end_op();
306          return -1;
307      }
308
309      iupdate(ip);
310      iunlock(ip);
311
312      end_op();
313      return 0;
314  }
315

```

심볼릭링크 시스템콜을 위한 레포 함수입니다.

create() 함수를 이용하여 inode를 새롭게 받아온 후, 리다이렉팅을 위한 old path 정보를 자신의 파일 정보에 저장합니다.

```
240
241 static struct inode*
242 create(char *path, short type, short major, short minor)
243 {
244     struct inode *ip, *dp;
245     char name[DIRSIZ];
246
247     if((dp = nameiparent(path, name)) == 0)
248         return 0;
249     ilock(dp);
250
251     if((ip = dirlookup(dp, name, 0)) != 0){
252         iunlockput(dp);
253         ilock(ip);
254         if(type == T_FILE && ip->type == T_FILE) // file이 이미 존재할 경우.
255             return ip;
256         if(type == T_SYM && ip->type == T_SYM) // sym 파일이 이미 존재할 경우.
257             return ip;
258         iunlockput(ip);
259         return 0;
260     }
261 }
```

create 파일의 경우 타겟 심볼릭 파일이 이미 존재하는 경우를 핸들링하는 로직을 추가합니다.

- stat.h

```
2023_eies021_2020003309 / project03 / C / stat.h
1  #define T_DIR  1  // Directory
2  #define T_FILE 2  // File
3  #define T_DEV  3  // Device
4  #define T_SYM  4  // Symbolic
5
```

심볼릭 파일을 구분하기 위해 새로운 inode 타입을 정의합니다.

- fs.c

```

681
682 struct inode*
683 trace_symbolic(struct inode *ip)
684 {
685     if (ip == 0) return 0;
686
687     uint binum;
688     char sym_path[128]; // sh.c main의 buf가 100이기 때문에 path가 100이 넘을 일은 없다.
689
690     while(ip->type == T_SYM) {
691         // symbolic link일 경우 redirection 합니다.
692         ilock(ip);
693         readi(ip, sym_path, 0, ip->size);
694         iunlock(ip);
695
696         cprintf("symbolic!!\n");
697         cprintf("(inum:%d) (size:%d) (sympath:%s)\n", ip->inum, ip->size, sym_path);
698
699         binum = ip->inum;
700         if((ip = namei(sym_path)) == 0) // 유효하지 않은 폴더
701             return 0;
702         if (binum == ip->inum) // 순환 링크인 경우
703             return 0;
704     }
705     // cprintf("get target file (inum:%d)!!\n\n", ip->inum);
706     return ip;
707 }
708

```

trace_symbolic 함수를 추가합니다.

해당 함수는 매개변수로 받은 inode가 심볼릭 파일일 경우, 타겟 파일까지 리다이렉팅을 해주는 함수입니다.

```

743
744 struct inode*
745 namei(char *path)
746 {
747     char name[DIRSIZ];
748     return trace_symbolic(namei(path, 0, name));
749 }
750

```

namei를 사용하는 많은 함수들은 심볼릭 링크가 가리키는 타겟 파일을 받아오기 위해 namei를 사용합니다. 따라서 trace_symbolic을 해당 함수에 추가해줍니다.

```

750
751 struct inode*
752 namei_ntr(char *path)
753 {
754     char name[DIRSIZ];
755     return namei(path, 0, name);
756 }
757

```

하지만 해당 path에 존재하는 파일의 메타데이터를 확인하기 위해 namei를 호출하는 일부 함수들이 존재합니다. ls 함수가 호출하는 stat 함수가, open 함수를 호출하는 경우가 그러합니다.

- ulib.c


```

70 int
71 stat(const char *n, struct stat *st)
72 {
73     int fd;
74     int r;
75
76     fd = open(n, O_RDONLY | O_NTRSYM);
77     if(fd < 0)
78         return -1;
79     r = fstat(fd, st);
80     close(fd);
81     return r;
82 }

```

stat가 open 함수를 호출하는 경우 메타데이터를 확인하기 위해 호출하는 경우입니다.

이를 구분하기 위해, O_NTRSYM이라는 플래그를 하나 만들어서 open 함수로 넘겨줬습니다.

- fcntl.h

```

4 #define O_CREATE 0x200
5 #define O_NTRSYM 0x010 // no trace symbolic

```

해당 플래그는 symbolic 파일을 만날 경우 trace를 하지 말란 의미입니다.

- sysfile.c

```

316 int
317 sys_open(void)
318 {
319     char *path;
320     int fd, omode;

```

```

334 }
335 } else {
336     if(omode & O_NTRSYM) ip = namei_ntr(path);
337     else ip = namei(path);
338
339     if(ip == 0){

```

```

342 }
343 ilock(ip);
344 if(ip->type == T_DIR && (omode != O_RDONLY && omode != O_NTRSYM)){
345     iunlockput(ip);
346     end_op();
347     return -1;
348 }

```

sys_open 함수를 편집합니다.

O_NTRSYM 플래그의 경우 namei_ntr 함수를 통해 inode를 받아옵니다.

또한 기존의 로직의 정상적인 흐름을 위해, 기존의 omode != O_RDONLY 코드가 omode != O_RDONLY && omode != O_NTRSYM 로 바뀐 변경사항이 존재합니다.

Sync

기존의 bcache 시스템을 이용합니다.

- bio.c

```
28
29 struct {
30     struct spinlock lock;
31     struct buf buf[NBUF];
32
33     // Linked list of all buffers, through prev/next.
34     // head.next is most recently used.
35     struct buf head;
36     int unused;
37     int bflush;
38 } bcache;
39
```

버퍼가 꽉 차있는지 확인하기 위해 unused 변수를 추가합니다.

현재 버퍼 flush가 진행 중인지 판별하기 위해 bflush 변수를 추가합니다.

```
40 void
41 binit(void)
42 {
43     struct buf *b;
```

```
57 }
58
59 bcache.unused = NBUF;
60 }
61
```

binit에서 사용되지 않은 버퍼의 개수는 NBUF로 시작합니다.

```
64 // In either case, return locked buffer.
65 static struct buf*
66 bget(uint dev, uint blockno)
67 {
68     struct buf *b;
69
70     acquire(&bcache.lock);
71
72     if (bcache.bflush) sleep(&bcache, &bcache.lock);
73
74     // Is the block already cached?
75     for(b = bcache.head.next; b != &bcache.head; b = b->next){
76         if(b->dev == dev && b->blockno == blockno){
77             if(b->refcnt == 0 && (b->flags & B_DIRTY) == 0)
78                 bcache.unused--;
79             b->refcnt++;
80             release(&bcache.lock);
81             acquire(&b->lock);
82             return b;
83         }
84     }
85 }
```

```

85
86     if (bfull()) bflush();
87
88     // Not cached; recycle an unused buffer.
89     // Even if refcnt==0, B_DIRTY indicates a buffer is in use
90     // because log.c has modified it but not yet committed it.
91     for(b = bcache.head.prev; b != &bcache.head; b = b->prev){
92         if(b->refcnt == 0 && (b->flags & B_DIRTY) == 0) {
93             bcache.unused--;
94             b->dev = dev;
95             b->blockno = blockno;
96             b->flags = 0;
97             b->refcnt = 1;
98             release(&bcache.lock);
99             acquiresleep(&b->lock);
100             return b;
101         }
102     }
103     panic("bget: no buffers");
104 }

```

bget 함수의 경우, sync system call을 제외하고 bflush를 호출하는 유일한 함수입니다.

해당 함수는, 현재 bflush가 진행 중이라면 sleep을 하여 bflush가 완료될 때까지 대기합니다.

또한 만약 cache에 블록이 없다면 unused 버퍼를 골라서 재활용하는 로직 사이에 변경 사항이 존재합니다. 만약 사용 가능한 unused 버퍼가 없다면, flush를 한차례 진행한 후 사용 가능한 unused 버퍼를 찾습니다. 이때, flush되는 버퍼의 기준은 bflush 함수 부분에서 더 자세히 보겠습니다.

bcache.unused 변수는 현재 참조 중인 프로세스가 없고, Dirty 플래그가 세워져 있지 않은 버퍼들의 개수를 의미합니다. 따라서 bget 함수는 unused 변수를 하나 감소 시킵니다.

```

130 // Move to the head of the MRU list.
131 void
132 brelease(struct buf *b)
133 {
134     if(!holdingsleep(&b->lock))
135         panic("brelease");
136
137     releasesleep(&b->lock);
138
139     acquire(&bcache.lock);
140
141     if (bcache.bflush) sleep(&bcache, &bcache.lock);
142
143     b->refcnt--;
144     if (b->refcnt == 0) {
145         // no one is waiting for it.
146         b->next->prev = b->prev;
147         b->prev->next = b->next;
148         b->next = bcache.head.next;
149         b->prev = &bcache.head;
150         bcache.head.next->prev = b;
151         bcache.head.next = b;
152         if((b->flags & B_DIRTY) == 0)
153             bcache.unused++;
154     }
155     release(&bcache.lock);
156 }
157 //PAGEBREAK!

```

반대로 버퍼에 대한 참조를 해제하는 함수인 brelease는 해당 버퍼가 조건에 부합한 경우, unused를 하나 증가시킵니다.

해당 함수의 경우, MRU list를 위해 버퍼의 위치를 조정합니다. 버퍼의 위치를 조정하는 것은, bflush 과정에서 문제가 있을 수 있기 때문에, bflush 시 대기해주는 로직을 추가합니다.

```

204
205 int
206 bfull()
207 {
208     // struct buf *b;
209     // int real_unused = 0;
210     // for(b = bcache.head.prev; b != &bcache.head; b = b->prev){
211     //     if(b->refcnt == 0 && (b->flags & B_DIRTY) == 0) {
212     //         real_unused++;
213     //     }
214     // }
215     // cprintf("bcache.unused : %d, real_unused : %d\n", bcache.unused, real_unused);
216
217     return bcache.unused == 0;
218 }

```

bfull 함수는 간단한 버퍼가 꽉 차있는지 확인하는 함수입니다.

```

160 int
161 bflush()
162 {
163     struct buf *b;
164     int flushed = 0, released = 0;
165
166     // cprintf("bflush\n");
167
168     if (!holding(&bcache.lock)) {
169         acquire(&bcache.lock);
170         released = 1;
171     }
172
173     if (bcache.bflush) sleep(&bcache, &bcache.lock);
174
175     bcache.bflush = 1;
176
177     for(b = bcache.head.next; b != &bcache.head; b = b->next){
178         if (b->refcnt != 0) continue;
179
180         release(&bcache.lock);
181         acquire(&b->lock);
182
183         if(b->flags & B_DIRTY && b->refcnt == 0){
184             iderw(b); // disk synk
185             b->flags &= ~B_DIRTY; // dirty flag clear
186             flushed++; // flush buffer inc
187         }
188         bcache.unused++;
189
190         releasesleep(&b->lock);
191         acquire(&bcache.lock);
192     }
193
194     // cprintf("\n\nbflush end\n\n");
195
196     bcache.bflush = 0;
197
198     wakeup(&bcache);
199
200     if(released) release(&bcache.lock);
201
202     return flushed;
203 }

```

버퍼를 디스크에 flush 시켜주는 함수입니다.

해당 함수는 bflush가 여러번 불릴 경우를 대비해 sleep, wakeup 로직을 통해 atomic 하게 동작하도록 합니다.

해당 함수가 flush 시키는 버퍼는 다음과 같습니다.

- $b \rightarrow \text{refcnt} == 0$
- $b \rightarrow \text{flags} \& B_DIRTY$ is True

해당 함수는 데드락을 방지하기 위해 현재 참조 중인 프로세스가 존재하는 버퍼는 flush 시키지 않습니다.

```

480
481     int
482     sys_sync(void)
483     {
484         return bflush();
485     }

```

sync system call은 bflush를 그대로 호출해서 리턴합니다.

특이 사항이 존재합니다. bflush는 panic 이 뜨거나 flush block num을 정상적으로 return 해주는 함수이기에 **실패 시 -1을 반환하는 경우는 없습니다.**

Result

Multi Indirect

```

34
35     void
36     createTestFile(uint filesize)
37     {
38         int i;
39         int idx = fd_idx++;
40
41         char filename[] = "testFile .txt";
42         filename[8] = (char)('0' + idx);
43
44         printf(stdout, "Create Test %d Started!\n", idx);
45
46         if((fds[idx] = open(filename, O_CREATE | O_RDWR)) < 0) {
47             printf(stdout, "error: creat file failed!\n");
48             exit();
49         }
50
51         const uint rep = filesize / 32;
52         for(i = 0; i < rep; i++) {
53             if(write(fds[idx], "aaaaaaaaaaaaa\n", 16) != 16) {
54                 printf(stdout, "error: write aa %d new file failed\n", i);
55                 exit();
56             }
57             if(write(fds[idx], "bbbbbbbbbbbbbbb\n", 16) != 16) {
58                 printf(stdout, "error: write bb %d new file failed\n", i);
59                 exit();
60             }
61         }
62
63         printf(stdout, "Create Test %d Finished!\n", idx);
64     }
65

```

```

104     void
105     create_test(void)
106     {
107         createTestFile(1 KB);
108         createTestFile(4 KB);
109         createTestFile(15 KB);
110         createTestFile(1 MB);
111         createTestFile(16 MB);
112
113         closeAllFDs();
114     }
115

```

```

65
66 void
67 readTestFile(int idx)
68 {
69     int i;
70     char filename[] = "testFile .txt";
71     filename[8] = (char)('0' + idx);
72
73     printf(stdout, "Read Test %d Started!\n", idx);
74
75     if((fds[idx] = open(filename, O_RDONLY)) < 0) {
76         printf(stdout, "error: open file failed!\n");
77         exit();
78     }
79
80     while((i = read(fds[idx], buf, sizeof(buf)))) {
81         for(int j = 0; j < i; j += 32) {
82             if(strncmp("aaaaaaaaaaaaa\nbbbbbbbbbbbbbb\n", &buf[j], 16)) {
83                 printf(stdout, "error: incorrect value detected at %d!\n", j);
84                 printf(stdout, &buf[j]);
85                 return;
86             }
87         }
88     }
89
90     printf(stdout, "Read Test %d Finished!\n", idx);
91 }
92

```

```

115
116 void
117 read_test(void)
118 {
119     readTestFile(0);
120     readTestFile(1);
121     readTestFile(2);
122     readTestFile(3);
123     readTestFile(4);
124
125     closeAllFDs();
126
127     removeTestFile(0);
128     removeTestFile(1);
129     removeTestFile(2);
130     removeTestFile(3);
131     removeTestFile(4);
132 }
133

```

Symbolic Link

```

cpu0: starting 0
sb: size 2100000 nblocks 2099339 ninodes 200 nlog 120 logstart 2 inodestart 122 bmap start8
init: starting sh
$ echo hello > file
$ mkdir dir
$ echo hello2 > dir/file2
$ cat file
hello
$ cat dir/file2
hello2
$ ln -s file link1
$ ln -s link1 link2
$ ln -s dir/file2 link3
$ cat link1
symbolic!!
(inum:22) (size:5) (sympath:file)
hello
$ cat link2
symbolic!!
(inum:23) (size:6) (sympath:link1)
symbolic!!
(inum:22) (size:5) (sympath:file)
hello
$ cat link3
symbolic!!
(inum:24) (size:10) (sympath:dir/file2)
hello2
$ rm file
$ cat link1
symbolic!!
(inum:22) (size:5) (sympath:file)
cat: cannot open link1
$ cat link2
symbolic!!
(inum:23) (size:6) (sympath:link1)
○ symbolic!!
(inum:22) (size:5) (sympath:file)
cat: cannot open link2
$ ln -s link3 link1
$ cat link1
symbolic!!
(inum:22) (size:6) (sympath:link3)
symbolic!!
(inum:24) (size:10) (sympath:dir/file2)
hello2
$ █

```

잘 되는 모습이다.

Sync

[illegible]

Multi Indirect Test 과정에서 사용한 코드를 사용해서 버퍼가 꽉 찰 때마다 bflush가 잘 실행됨을 알 수 있었다.

Trouble Shooting

- Sync 구현 후 xv6 켜다 킬 시, 생성했던 파일들이 사라짐.

시간이 부족하여 정확한 원인을 파악하지 못했다.

echo와 ln을 사용하여 여러 파일들을 생성한 후 xv6를 꺾다 킬 시, 생성했던 파일들이 없다.

```
init: starting sh
$ ls
.          1 1 512
..         1 1 512
README    2 2 2286
cat       2 3 15544
echo      2 4 14424
forktest  2 5 8860
grep      2 6 18380
init      2 7 15048
kill      2 8 14508
ln        2 9 14700
ls        2 10 16976
mkdir     2 11 14532
rm        2 12 14512
sh        2 13 28564
stressfs  2 14 15440
wc        2 15 15960
zombie    2 16 14080
indirect_test 2 17 20856
console   3 18 0
$
```

Multi Indirect & Sync를 테스트하고 난 뒤 삭제한 테스트 파일들이 xv6를 꺾다 킬 시, 다시 생성된다.

```
$ ls
.          1 1 512
..         1 1 512
README    2 2 2286
cat       2 3 15580
echo      2 4 14456
forktest  2 5 8892
grep      2 6 18416
init      2 7 15080
kill      2 8 14544
ln        2 9 14732
ls        2 10 17012
mkdir     2 11 14564
rm        2 12 14544
sh        2 13 28596
stressfs  2 14 15476
wc        2 15 15992
zombie    2 16 14116
indirect_test 2 17 20888
console   3 18 0
testFile0.txt 2 19 1024
testFile1.txt 2 20 4096
testFile2.txt 2 21 15360
testFile3.txt 2 22 1048576
testFile4.txt 2 23 16769040
$
```

- bflush Deadlock

bflush가 참조하는 버퍼에 어떤 프로세스가 참조하고 있을 경우, bflush가 `acquiresleep(&b->lock);`을 호출할 때 깨워줄 프로세스가 깨워줄 수 없어서 생기는 문제로, `if (b->refcnt != 0) continue;` 로직을 추가하여 해결했다.

```
176
177     for(b = bcache.head.next; b != &bcache.head; b = b->next){
178         if (b->refcnt != 0) continue;
179
180         release(&bcache.lock);
181         acquiresleep(&b->lock);
182
183         if(b->flags & B_DIRTY && b->refcnt == 0){
184             iderw(b); // disk synk
185             b->flags &= ~B_DIRTY; // dirty flag clear
186             flushed++; // flush buffer inc
187         }
188         bcache.unused++;
189
190         releasesleep(&b->lock);
191         acquire(&bcache.lock);
192     }
193
```

- file의 변경 내용 반영

sync를 호출하지 않아도, file의 변경 내용이 반영되는 것을 확인할 수 있었다. 이는 시간이 부족하여 미처 핸들링하지 못한 오류이며 원인은 다음의 내용으로 추정된다.

1. sync를 호출하지 않고 file close 시, 정상적으로 변경 내용이 반영되지 않는다. 하지만 버퍼에는 해당 변경 사항이 그대로 존재한다. 따라서 이후 추가적인 행동을 할 경우, 파일의 변경 사항이 담긴 버퍼의 내용이 flush 되는 경우가 존재한다.
2. 파일 사이즈는 ls 명령어로 확인했다. 해당 정보는 파일의 메타데이터이므로 버퍼의 flush를 막는게 아니라, dinode 업데이트를 막아야 한다. 따라서 `writel`와 `iupdate` 함수를 자세히 살펴보고 관련 로직을 변경해야 한다.